



Intro to Backend: API's + Flask

with Maddie and Kaavya

**Before we get started, some
announcements...**

Linktree!

- Newsletter
- Website
- Instagram
- GitHub
- And more...



Discord
exclusive



Hack for Humanity



ACM & ACM-W PRESENT
13th Annual:

The logo for Hack for Humanity is centered. It features a stylized globe with a yellow outline. Overlaid on the globe is a yellow banner with the text '< HACK FOR >' in a small, sans-serif font, and 'HUMANITY' in a large, bold, sans-serif font below it. The background of the poster is a dark blue space scene with a bright yellow sun or starburst in the upper left, casting rays across the scene, and a mountain range visible in the lower half.

**SANTA CLARA UNIVERSITY'S HUMANITARIAN
HACKATHON**

February 28th - March 1st, 2026
Heafey Hall

Join us in solving
pressing social
issues using your
creativity and
code!

A square QR code is located to the right of the text 'Join us in solving...'. It has a yellow center with a stylized '@' symbol and a circular arrow around it.

tinyurl.com/H4H-sign-up

Powered By the Ciocca Center

Registration deadline is Friday, 2/27 at 11:59PM PT

In compliance with ADA/504, please direct your accommodation requests to jesperson@scu.edu

CyberSisters Program



- Year Long Program!
- Both Mentors and Mentees are needed
- Open year round



Other Events



Cyber Sisters Outing!

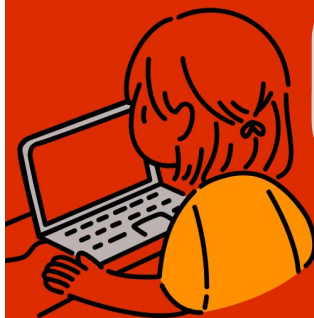
Thurs 1/22
5:30pm



FULL-STACK MINI SERIES

Firestore WORKSHOP

Backend + Database Storage



Thursday 1/22
7:00pm
SCDI 2302

In compliance with ADA/504, please direct your accommodation requests to jesperson@scu.edu

BRONCOSEC

STORING IMAGE SECRETS: GETTING YOUR 1000 WORDS WORTH

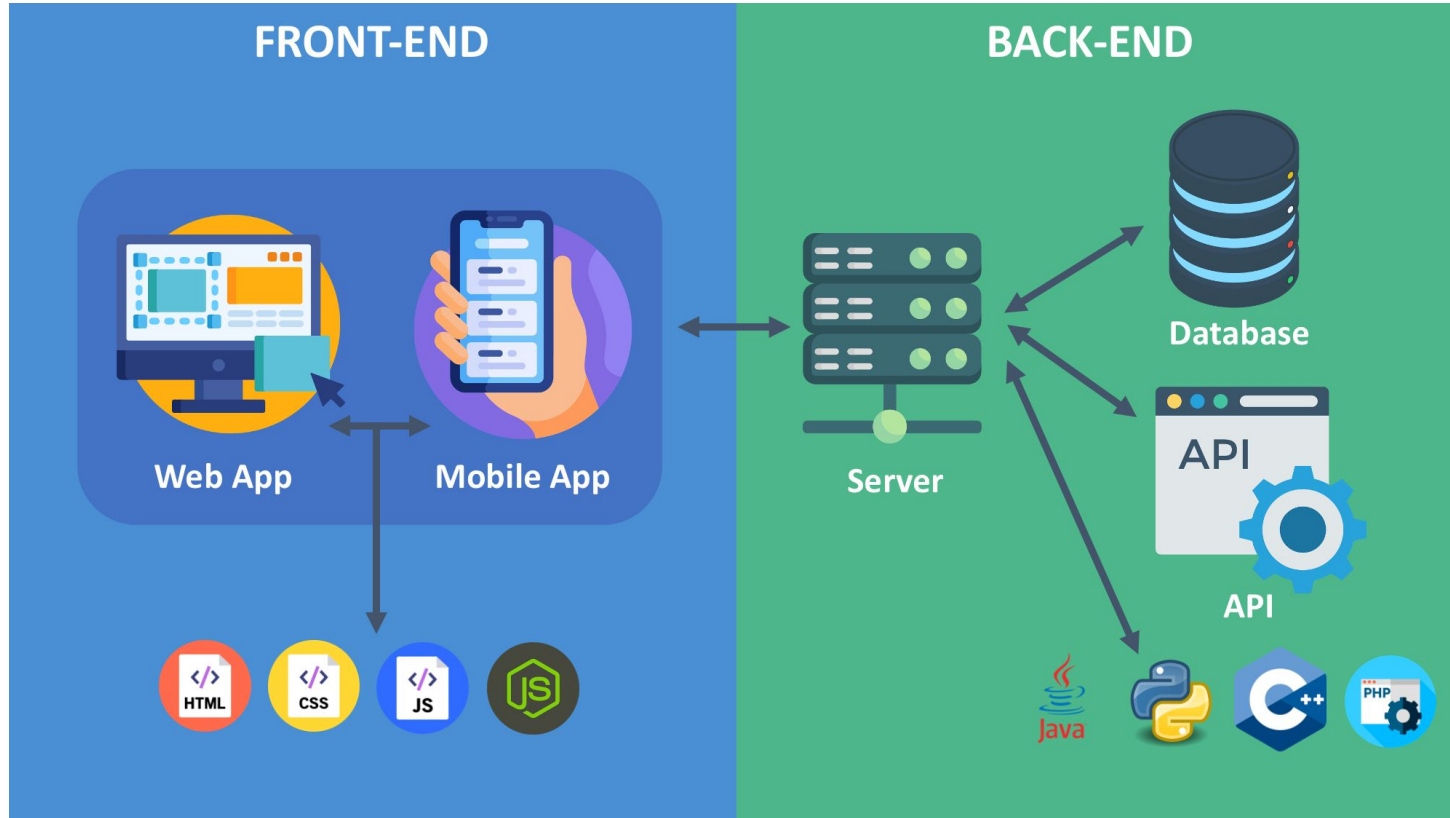
Prepare for BroncoCTF by learning about the all ways images can carry secrets, from using steganography to hide data within pixels to storing key metadata like location information.

MONDAY JAN 26
5:45PM
»»» KENNA 104

In compliance with ADA/504, please direct your accommodation requests to ACM BroncoSec at BroncoSecClub@scu.edu at least 72 hours prior to the event.

Now for the workshop...

The Big Picture



What is the backend?

- The part of an application that handles logic, data, and security
- Usual components: server, application code, database, API's
- User's can't see it but every user action uses it

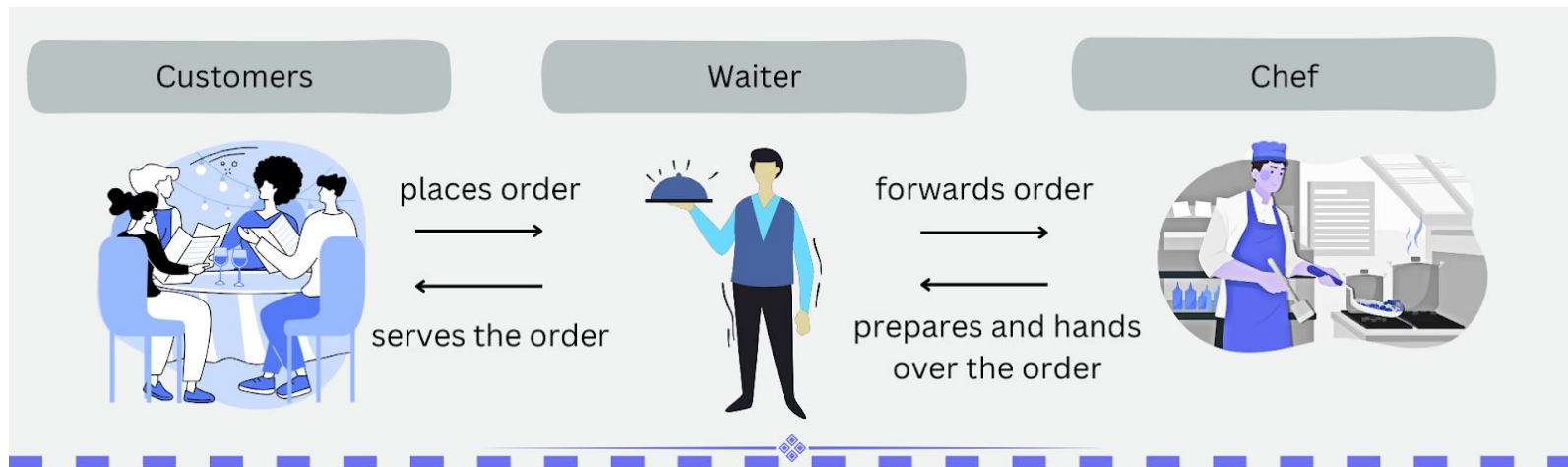
Why do I need backend?

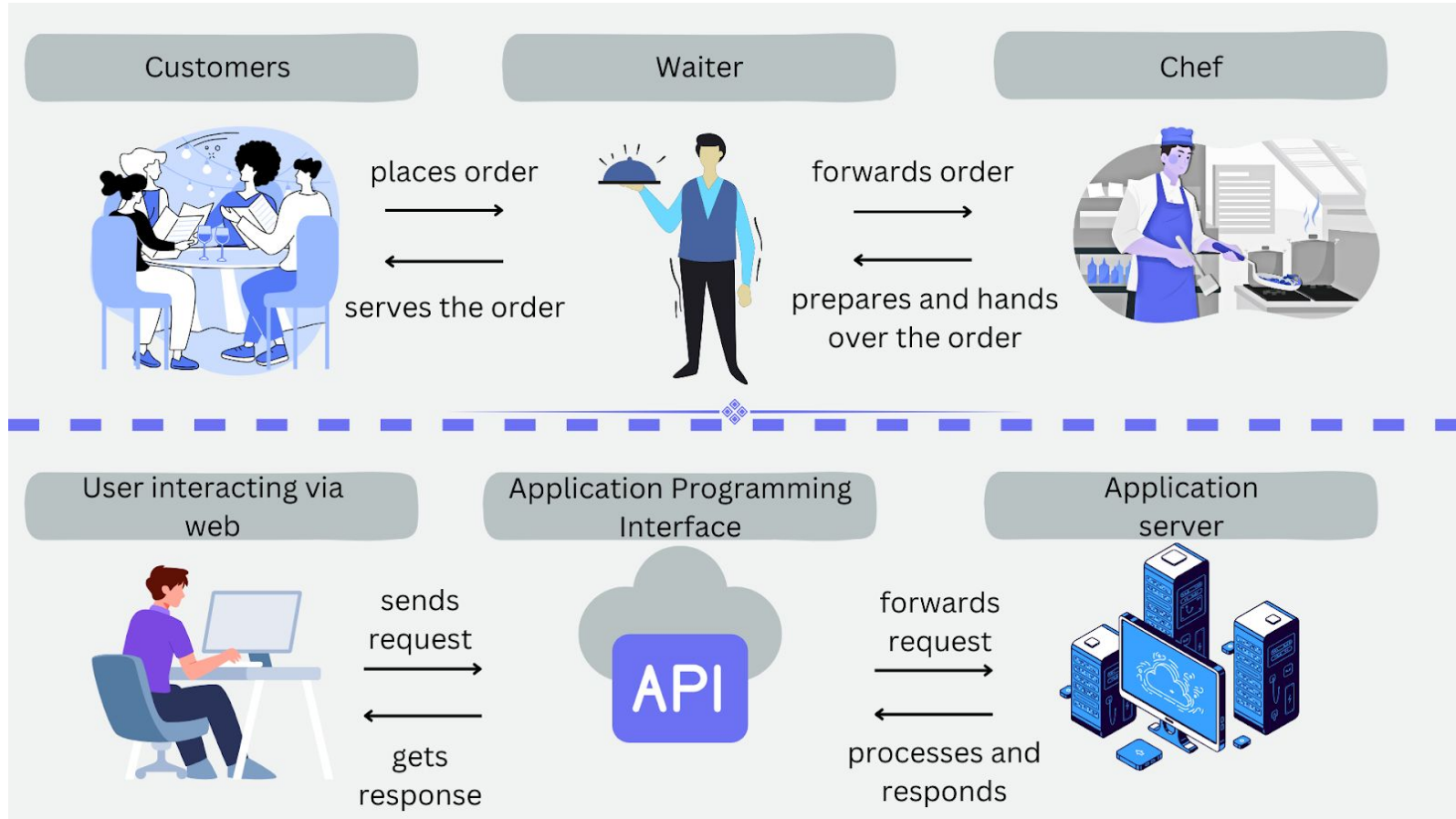
- **Every non-trivial app needs shared state:** user accounts, sessions, permissions, and data cannot live safely in the frontend.
- **Security boundary:** secrets, API keys, and authorization logic must run on a trusted server, not in the client.
- **Concurrency:** thousands of users may read/write the same data at the same time, coordination requires a backend.
- **Persistence:** long-term storage and recovery require a server-side system.

API's

What is an API?

- Application Programming Interface
- An API is how one program talks to another program.
- It tells you what you can ask for and how to ask for it.
- The frontend uses the API to request data or actions from the backend.
- The backend sends back a response.





HTTP Methods

- HTTP methods tell the server what action you want to do with a resource.
- Common ones:
 - **GET** – get data
 - **POST** – create new data
 - **PUT** – update existing data
 - **DELETE** – remove data

JSON



- Javascript Object Notation
- Format for sending data between a frontend and backend
- Easy for humans to read and for machines to parse.
- Most APIs send and receive data in JSON.

Example

```
'{"name":"John", "age":30, "car":null}'
```

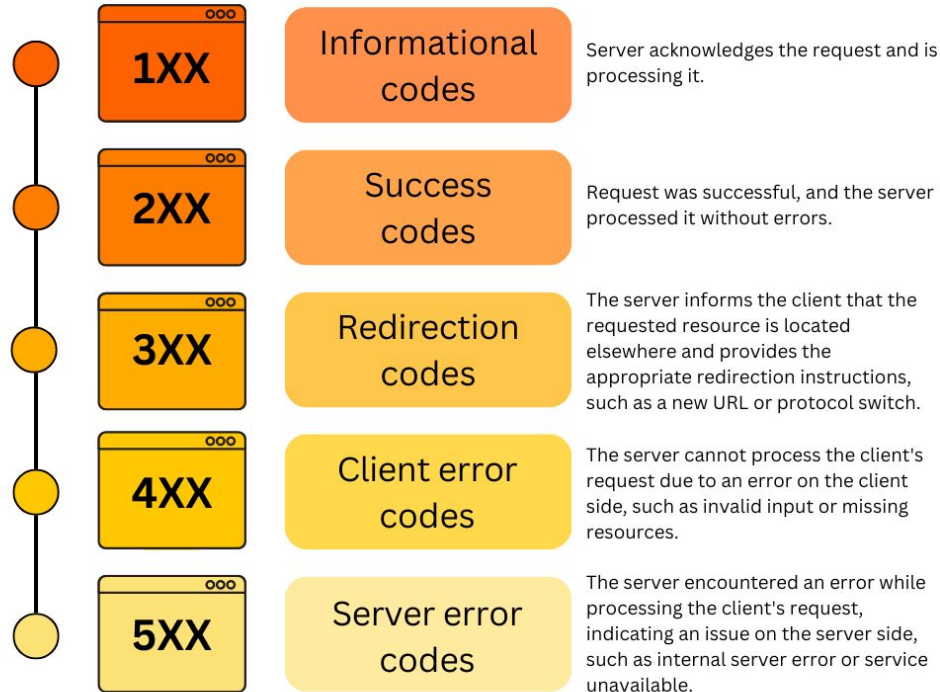
What is a REST API

- Representational State Transfer API
- A REST API is an API that follows a simple set of web rules called REST.
- It uses standard HTTP methods: GET, POST, PUT, DELETE.
- Requests are sent from client to server via a Web URL using one of the HTTP methods
- Each URL represents a resource (users, posts, orders).
- It is stateless: each request contains all the information needed.

Can't Front End Handle API Requests?

- Yes... **BUT** in a very limited capacity
- **Browsers are not trustworthy because any code and keys can be inspected by users.**
- The backend acts as the secure gatekeeper between the frontend and data/services.
- The frontend should not contain any sensitive information

HTTP Status Codes



API Response

HTTP status code

HTTP/1.1 404 Not Found

Content-Type: application/json;charset=UTF-8

Content-Length: 72

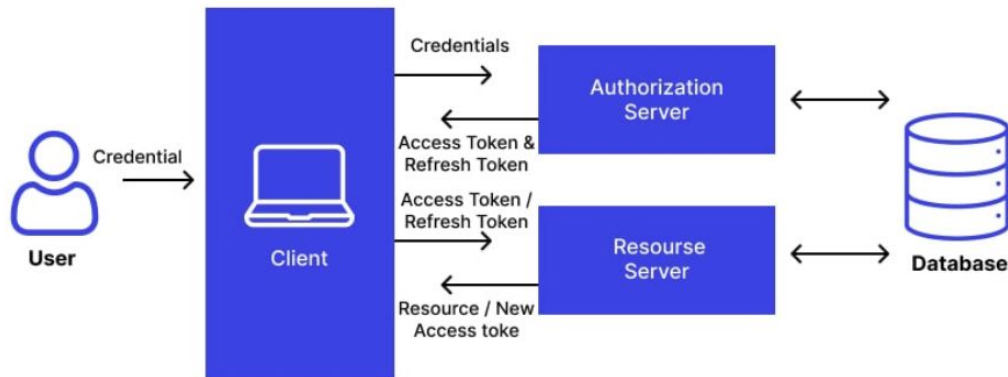
HTTP headers

```
{
  "errorCode": 1020,
  "message": "User not found.",
  "refId": "ookmfyguq538"
}
```

Body

Tokens

- Tokens are digital keys that prove your identity to a server.
- Used for authentication (who you are) and authorization (what you can do).
- Common types: JWT (JSON Web Token) and API keys.
- The server checks the token before letting you access data or actions.



- CORS (Cross-Orin Resource Sharing) is a mechanism that allows a server to indicate any origins besides its own from which a browser can load resources.
- More simply, it controls which websites can talk to your backend.
- If your frontend and backend are on different domains, the browser checks CORS before allowing the request.
- The backend must explicitly allow trusted origins.

Let's Practice!



Simple Demo w/
<https://jsonplaceholder.typicode.com/>

Flask

What is Flask?

Flask is a lightweight Python web framework used to build:

- Web servers
- Backend applications
- APIs

It lets Python respond to HTTP requests (like GET, POST, PUT, DELETE) from browsers or front-end apps. Without Flask, our Python code could not talk to the web.

Flask gives us:

- A running web server
- URL routing
- Request handling



Demo: CRUD

Steps



1. Setup
 - a. Install Python
 - b. Virtual Environment Setup
 - c. Install Flask
2. Write API's
3. Integrate with React Front End
 - a. Build React app
 - b. Edit Flask instance

Setup



Create project directory, cd into it

Install Python

Python: <https://www.python.org/downloads/> pip: <https://pip.pypa.io/en/stable/installation/>

```
python --version
```

```
pip --version
```

Virtual Environment Setup

Creating: `python -m venv .venv`

Activating: `source .venv/bin/activate` Deactivating: `deactivate`

Install Flask

```
pip install flask or pip install -r requirements.txt
```

Keep in Mind + Next Steps

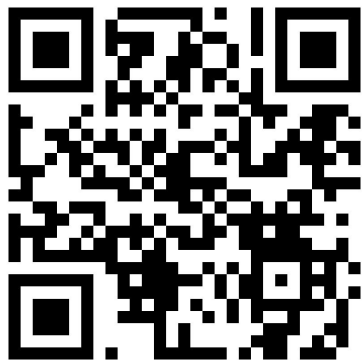
- We used simple data locally in our file
 - This would be replaced with querying a database for a real project
- Development settings (like debug=True and open CORS) should be disabled or restricted in production.
- APIs should include authentication and authorization before exposing real data.
 - Ex. API keys, tokens, or login-based access so only approved users can call certain endpoints (Use environment variables for secrets and configuration → never hardcode keys or passwords).
- Add basic error handling so the API fails safely and predictably
- Keep venv in .gitignore

Questions?

Kahoot!!



Linktree!



Discord

Thank you!!

